Use Case 1 — Data Enrichment

Dynamic, Schema-Driven Pipeline with Two-Agent Architecture

What exists vs. what's claimed

Pipeline currently in place

```
Raw CSV
  │
  ├── Step 1  Load raw data, inspect schema + null rates
  ├── Step 2  Identify exact-match duplicate groups (on description)
  ├── Step 3  Strip whitespace
  ├── Step 4  Lowercase brands
  ├── Step 5  Remove noise words (legal suffixes + frequency analysis +
aliases dict)
  ├── Step 6  Strip punctuation
  ├── Step 7  Regex: extract sizes → sizes column, scan ingredients →
allergens column
  ├── Step 8  Blocking + fuzzy matching (rapidfuzz, weights: name×0.5
brand×0.2 combined×0.3, threshold ≥85)
  ├── Step 9  Union-Find clustering (transitive closure)
  ├── Step 10 Golden record selection (Completeness×0.4 + Freshness×0.35 +
IngredientRichness×0.25)
  ├── Step 11 LLM enrichment via Groq/Llama 3.3 70B (4-layer:
dedup→rules→cache→batch)
  └── Step 12 Final output — downloadable CSV
```

Known issues (carried forward — not yet fixed)

- **LLM calls not firing** — `llm_rows=0` bug in `enrich_row`, checking wrong variable
- **Dedup runs after enrichment** — LLM called on rows that get dropped. Correct order: clean → dedup → merge → enrich
- **Merge strategy is lossy** — picks one row, drops rest. Column-wise merge not implemented
- **All thresholds hardcoded** — no rules layer exists
- **Domain-locked** — quantity stripping hardcoded, breaks for non-nutrition domains

What's Being Built (Revamp)

Core design shift

The old model: a fixed 12-step script where every behavior is hardcoded for USDA nutrition data.

The new model: **the pipeline is defined by a unified output schema**. The system reasons about how to get any incoming data source to conform to that schema. When it encounters a transformation it doesn't know how to do, two agents generate and persist the missing function. The pipeline grows its own capabilities over time.

Full Architecture

```
USER CONNECTS DATA SOURCE
         │
         ▼
┌──────────────────────────────────────────────────────────┐
│               AGENT 1 — Orchestrator                     │
│                                                          │
│  Access:                                                 │
│  ├── File system (block registry, saved functions)       │
│  ├── Incoming CSV (schema, sample rows, null rates)      │
│  ├── User requirements (domain, extra columns)           │
│  └── Unified output schema                               │
│                                                          │
│  LLM CALL — Gap Analysis                                 │
│  Input:  incoming schema + sample values                 │
│          unified output schema                           │
│  Output: mapping decision per column —                   │
│    MAP:  source_col → unified_col                        │
│    DROP: source_col (no equivalent)                      │
│    NEW:  source_col → suggested unified name             │
│    ADD:  unified_col missing entirely from source        │
└──────────────────────────────────────────────────────────┘
                │
                │ gap analysis report
                ▼
┌──────────────────────────────────────────────────────────┐
│           HITL REVIEW 1 — Column Mapping                 │
│                                                          │
│  User sees every mapping decision Agent 1 made:          │
```

```
│  ├── product_description → product_name   [MAP]     │
│  ├── recalling_firm      → brand_name     [MAP]     │
│  ├── voluntary_mandated  → (drop)         [DROP]    │
│  └── allergens           → allergens      [ADD]     │
│                                                      │
│  User can override any row before approving          │
│  [Approve] → Agent 1 passes approved mapping         │
│              to Agent 2                              │
└──────────────┬──────────────────────────────────────┘
               │ approved mapping + gaps
               ▼
┌──────────────────────────────────────────────────────┐
│          AGENT 2 — Code Generator                    │
│                                                      │
│  Access:                                             │
│  └── Approved mapping + context from Agent 1 only    │
│      (no file system, no network, no raw data)       │
│                                                      │
│  LLM CALL — Block Selection                          │
│  Input:  approved mapping gaps                       │
│          current block registry (names + desc)       │
│  Output: per gap —                                   │
│    ├── EXISTING BLOCK covers this → fetch and use    │
│    └── NO BLOCK covers this → generate new one       │
└──────────┬──────────────────────┬────────────────────┘
           │                      │
    [Block exists]         [Block missing]
           │                      │
           ▼                      ▼
    Fetch from           LLM CALL — Generate
    registry             transformation function
    Execute              ┌──────────────────────┐
    directly             │ HITL REVIEW 2        │
                         │ — New Block Approval │
                         │                      │
                         │ User sees:           │
                         │ ├── Function code    │
                         │ ├── Sample I/O       │
                         │ └── Validation result│
                         │                      │
                         │ [Approve] → execute  │
                         │ [Reject]  → re-prompt│
                         └──────────┬───────────┘
                                    │ approved
                                    ▼
                              Execute in Docker
```

```
                              sandbox (subprocess,
                              no network, no fs)
                                     │
                                     ▼
                              Save to block
                              registry for reuse
            │                     │
            └─────────────────────┘
                       │
                       ▼
┌──────────────────────────────────────────────────────┐
│               BLOCK RUNNER                           │
│                                                      │
│  Executes all blocks in sequence:                    │
│  ├── Fetched existing blocks                         │
│  └── Newly generated + approved blocks               │
└──────────────────────────────────────────────────────┘
                │
                │
                ▼
┌──────────────────────────────────────────────────────┐
│               ENRICHMENT LAYER                       │
│                                                      │
│  4-tier hierarchy (cheapest first):                  │
│  1. Deterministic rules (regex, keyword scan)        │
│  2. Embedding similarity to known taxonomy           │
│  3. Cluster label propagation (share across dedup)   │
│  4. LLM call — only for truly ambiguous rows         │
└──────────────────────────────────────────────────────┘
                │
                │
                ▼
┌──────────────────────────────────────────────────────┐
│        POST-ENRICHMENT HITL VALIDATOR                │
│                                                      │
│  Each row checked against unified schema contract    │
│  ├── Pass → written to output                        │
│  └── Fail → quarantined with reason tag              │
│               → human reviews quarantine log         │
└──────────────────────────────────────────────────────┘
```

Corrected Pipeline Step Order

Old (broken): clean → enrich → dedup → score

New (correct):

```
dq_score_pre
  → [Agent-generated transforms for schema gaps]
  → strip_whitespace
  → lowercase_brand
  → remove_noise_words
  → strip_punctuation
  → [domain-specific block: extract_quantity_column OR
keep_quantity_in_name]
  → extract_allergens
  → fuzzy_deduplicate
  → column_wise_merge        ← replaces lossy single-row selection
  → golden_record_select
  → llm_enrich               ← now runs only on unique post-dedup rows
  → dq_score_post
```

Dedup before enrich means LLM only runs on unique products. If 1000 rows collapse to 650 unique products, 350 fewer LLM calls.

Block Registry

Pre-built blocks (always available, no agent call needed):

Block ID	What it does	Domain
`strip_whitespace`	Remove leading/trailing whitespace from all string fields	All
`lowercase_brand`	Normalize brand casing	All
`remove_noise_words`	Strip legal suffixes + frequency-based noise + aliases dict	All
`strip_punctuation`	Replace non-alphanumeric with spaces	All
`extract_quantity_column`	Pull sizes out of name → separate column, save to `sizes`	Nutrition
`keep_quantity_in_name`	Leave quantity embedded in name	Pricing
`extract_allergens`	FDA Big-9 keyword scan of ingredients field	Nutrition/Safety
`fuzzy_deduplicate`	Blocking + rapidfuzz + union-find clustering	All
`column_wise_merge`	Best-value-per-field merge across duplicate cluster	All
`dq_score_pre`	Score completeness/freshness before enrichment	All

Block ID	What it does	Domain
`golden_record_select`	Pick canonical record per cluster by DQ composite score	All
`llm_enrich`	Groq LLM call — 4-tier hierarchy, only for ambiguous rows	All
`dq_score_post`	Re-score post-enrichment, compute delta, add DQ_delta column	All

Agent-generated blocks (created on demand, saved to registry):

Trigger	Example generated block
Type mismatch source → output schema	`string_to_float_price` — strips currency symbols, converts to float
Column exists but wrong format	`normalize_unit_ml_to_oz` — unit conversion with regex
New column derivable from existing field	`extract_recall_reason` — keyword classification from recall description
Domain switch with no registry coverage	Any novel transformation for a new domain

---

Function Registry (Persistent Memory)

Generated functions are saved, not discarded. Agent 1 checks the registry before calling Agent 2.

```
registry/
├── registry.json                          ← index of all saved functions
└── functions/
    ├── string_to_float_currency.py
    ├── extract_recall_reason.py
    └── normalize_unit_ml_to_oz.py
```

Each registry entry tracks: `source_type`, `target_type`, `tags`, `domain`, `used_count`, `validation_passed`, `created_at`.

Lookup logic: match on `source_type + target_type` first, use `tags` to rank among multiple matches. Registry hit = zero Agent 2 call, zero LLM cost. Over time the registry becomes a

domain-specific transformation library the pipeline built for itself.

Enrichment Hierarchy (replaces naive "call LLM on everything")

Tier	Method	Cost	Coverage
1	Deterministic rules (regex, FDA Big-9 scan, USDA category map)	Free	~60% of rows
2	Embedding similarity to known product taxonomy	Cheap (one-time embed)	~25% of rows
3	Cluster label propagation (if 2/3 cluster members have value, third inherits)	Free	~10% of rows
4	LLM call via Groq	Per-token cost	~5% of rows

Only rows that fall through tiers 1–3 reach the LLM. The claim: **"We use LLM surgically, not as a crutch."**

DQ Scoring Model

Two scores per row, both persisted as columns in the output:

Column	Description
`dq_score_pre`	Completeness + freshness score before any transformation
`dq_score_post`	Same formula applied after enrichment
`dq_delta`	`dq_score_post - dq_score_pre` — the enrichment's contribution

Score formula: `Completeness × 0.4 + Freshness × 0.35 + IngredientRichness × 0.25`

Weights are declared in the unified schema, not hardcoded — changing them requires only a schema edit.

Important caveat: Post-enrichment nulls are honest. The LLM fills structurally inferable fields (`primary_category` , `dietary_tags` , `is_organic`). Fields requiring source data that was

never provided remain null and are reflected in `dq_score_post`. The delta shows what enrichment actually contributed, not what we wished it did.

Demo Narrative

What the audience sees:

1. User connects USDA FoodData → pipeline runs → unified schema established → output shown
2. User connects FDA Recalls → Agent 1 detects schema gap → calls Agent 2 → function generated → saved to registry → pipeline runs → output matches unified schema
3. User switches domain to "pricing" → LLM Planner generates a different YAML (keep_quantity_in_name instead of extract_quantity_column) → same codebase, different behavior
4. Third run of any source → registry hits fire, Agent 2 never called → "the pipeline remembered"

The answer to "can this only handle nutritional data?": *"No. Connect any data source. Agent 1 detects what transformations are missing. Agent 2 generates them. The pipeline YAML is planned by an LLM based on the domain. Changing the domain changes the pipeline — not the code."*

What's Still Broken / Not Built

Item	Status
LLM enrichment (Groq) firing correctly	Bug not fixed — priority 1
Column-wise merge	Not implemented — priority 2
Two-agent system (Agent 1 + Agent 2)	Not built — demo being built separately
LLM Pipeline Planner	Not built
Function registry	Not built
Embedding similarity tier	Not built
Cluster label propagation	Not built
Post-enrichment HITL validator	Not built
DQ delta column	Not built

What Actually Exists

What UC1 incidentally produces that UC2 can consume:

- `dq_score_pre` and `dq_score_post` per row
- `enriched_by_llm` boolean per row
- `transformations_applied` list per row
- `duplicate_group_id`, `canonical`, `duplicate_score` per row

That's it. No anomaly detection, no streaming, no chatbot, no dashboard.

Known issues

- No audit log exists yet — planned in UC1 revamp but not built
- No time-series data exists — single pipeline run produces a snapshot, not a trend
- RPCA + Isolation Forest are listed in the tech stack but not implemented
- Pathway streaming is listed but not implemented
- ChromaDB for RAG chatbot not set up
- No schema for what the chatbot should be able to answer

What's Planned

Central Dashboard

A single screen with right-sidebar RAG chatbot. The left/center panel shows all diagnostic data. The right panel is the chatbot interface.

Diagnostic Data Sources (what populates the dashboard)

Per-run pipeline metrics (from UC1 audit log):

- Block execution trace: which blocks ran, in what order, rows in/out per block
- Null rate per field: before and after each block (shows exactly which block fixed what)
- DQ score distribution: histogram pre vs post enrichment, per data source
- LLM call log: rows that triggered LLM, confidence scores, fallback-to-rules count, cost per run
- Quarantine log: rows that failed post-enrichment schema validation, with reason tags
- Dedup log: cluster sizes, merge decisions, column-wise merge conflict resolutions
- Rules file hash: which YAML governed this run (reproducibility anchor)

Anomaly signals (Isolation Forest on per-run metrics):

- Null rate spike: field X jumped from 2% → 40% between runs
- Row count deviation: expected 1000 rows, got 847 — where did 153 go?
- DQ score distribution shift: median score dropped 12 points after source refresh
- LLM confidence degradation: average confidence fell below 0.6 threshold
- Duplicate rate anomaly: dedup rate jumped from 8% → 31% — source sent repeated data

Source-level health (per data source, per ingestion run):

- Last ingestion timestamp
- Row count vs. baseline
- Null rate per key field vs. baseline
- Schema drift detection: new columns appeared, expected columns missing

Cost tracking:

- LLM calls per run, tokens in/out, cost per run, cost per product enriched
- Cache hit rate: how many rows served from cache vs. fresh LLM call
- Trend over time: cost per run across last N runs

RAG Chatbot

Powered by ChromaDB + Claude. The document corpus is the audit logs, anomaly reports, and pipeline run history.

Example questions it should answer:

- *"Why did the March 28th run produce fewer rows than March 21st?"*
- *"Which block is responsible for the spike in null brand_owner values?"*
- *"How many products were quarantined in the last run and why?"*
- *"What did the LLM enrich that rules couldn't handle?"*
- *"Is the current DQ score distribution normal compared to the last 5 runs?"*

What's NOT being built (scope cut)

- Pathway real-time streaming — too complex for 3-week timeline, replaced by per-run batch metrics
- Alert system / email notifications — out of scope for demo
- Multi-pipeline monitoring — UC2 only observes the UC1 pipeline

---

UC3 + UC4 — Product Catalog

What Actually Exists

A **pre-computed tab** in `app.py` (the main Streamlit dashboard). Static data loaded from `POC/data/`. No live search, no live recommendations, no OpenSearch connection.

UC3 (Search): A search UI tab exists in the POC. It runs against pre-computed results — not a live OpenSearch query. BM25 + semantic hybrid search is listed in the tech stack but OpenSearch is not configured or running locally.

UC4 (Recommendations): A recommendations tab exists. It loads pre-computed Instacart co-purchase associations from `POC/data/`. The association rules were computed offline using mlxtend. No live query, no neo4j connection.

Known issues

- No live search index — demo runs against static files, not OpenSearch
- No before/after contrast — only enriched data is shown, raw catalog is not indexed
- Recommendations are not connected to the product being viewed — it's a static list
- No neo4j graph loaded — relationships exist as flat CSV, not a graph
- LLM-as-judge AutoEval (nDCG/MRR) is described in the doc but not implemented

---

What's Planned

Demo Entry Point — Raw Catalog First

Before showing enrichment, the Product Catalog screen opens on **raw USDA data**. This is the failure state the audience needs to see before UC1 makes sense.

Raw catalog demo moments:

- Search "spicy chicken sandwich" → zero results or irrelevant noise
- Search "cheerios" → returns 6 separate rows (20g box, 28g box, 26g box) as if they're different products
- View any product → recommendations are noisy (weak signal because co-purchase data is split across duplicate IDs)

Then: trigger UC1 enrichment. Observability Layer lights up. Pipeline runs.

Enriched catalog demo moments (same queries):

- Search "spicy chicken sandwich" → returns ranked, relevant results (LLM extracted `primary_category: "Poultry"`, normalized name)

- Search "cheerios" → returns 1 canonical record with all size variants listed in `sizes` column
- View Cheerios → cross-category recommendations are consolidated and coherent (co-purchase signal no longer split)

UC3 — Hybrid Search

Index: OpenSearch with two fields per product — BM25 over `clean_name + primary_category + dietary_tags`, dense vector over enriched text embedding.

Query flow:

```
User types query
    ↓
BM25 retrieval (keyword match)  +  Semantic retrieval (vector similarity)
    ↓
Reciprocal Rank Fusion (RRF) — merges both ranked lists
    ↓
Results displayed with: product name, brand, category, DQ score, allergens
```

Evaluation (LLM-as-Judge):

- 300–500 query/result pairs from Amazon ESCI benchmark
- Claude judges relevance: Exact / Substitute / Complement / Irrelevant
- nDCG and MRR computed before and after enrichment
- This is the quantified proof that UC1 improved search quality

The delta that matters: nDCG score on raw catalog vs. enriched catalog. If enrichment moved nDCG from 0.45 → 0.71, that's the slide. One number. Immediately understood.

UC4 — Recommendations

Data: Instacart 37.3M rows → association rules via mlxtend (support, confidence, lift). Runs on deduplicated product IDs — this is the key dependency on UC1.

What's shown per product:

- "Customers who bought this also bought:" → direct co-purchase associations
- "You might also like:" → cross-category affinity (e.g. Cheerios → Milk → Bananas)

The delta that matters: Pick one product (e.g. Doritos). In raw data: 6 product IDs, co-purchase signal split, weak associations, lift scores low. Post-enrichment: 1 canonical ID, signal consolidated, lift scores 3–4x higher, recommendations are coherent.

Show this as a side-by-side card. Left: raw recommendations (noisy, repetitive, low confidence). Right: enriched recommendations (cross-category, high lift, makes intuitive sense). That's the moment.

Neo4j (stretch):

- Products, brands, categories as nodes
- Co-purchase relationships as edges with lift score as weight
- Shortest path between distant categories = non-obvious recommendation
- If time allows: show graph visualization in the demo

Dynamic Pipeline -- 2 Agent Architecture

Schema-Driven, Self-Extending ETL with Isolated Code Execution

---

Core Idea

A static ETL pipeline is defined once and breaks when the data changes. This pipeline is **self-extending** — when a new data source introduces transformations that don't exist yet, two agents collaborate to generate, validate, and persist the missing functions. The pipeline grows its own capabilities over time without human code changes.

The Two Agents

Agent 1 — Context Collector (Orchestrator)

Role: Understands the problem. Feeds Agent 2 exactly what it needs — nothing more.

Has access to:

- File system (existing transformation library, saved function registry)
- Incoming data source (schema, sample rows, data types, null rates)
- User-defined requirements (additional columns requested, domain declaration)
- Output schema (the unified downstream schema all sources must conform to)

Does NOT generate code. Its job is to:

1. Diff the incoming source schema against the output schema — identify gaps
2. Check the function registry — does a saved function already cover this gap?

3. If yes → reuse it, no Agent 2 call needed
4. If no → construct a precise context package and hand off to Agent 2
5. Receive Agent 2's output, validate it, register it, wire it into the pipeline

Context package Agent 1 sends to Agent 2:

```
{
  "gap": {
    "target_column": "price_usd",
    "target_type": "float",
    "source_column": "price_raw",
    "source_type": "string",
    "sample_values": ["$1.99", "2.50 USD", "£3.00", "¥400"]
  },
  "constraints": {
    "must_handle_nulls": true,
    "output_null_on_failure": true,
    "no_external_libraries": true
  },
  "existing_functions": ["strip_whitespace", "lowercase_brand"],
  "domain": "pricing"
}
```

Agent 1 sends **only this** — no file system access, no web access, no raw data beyond samples.

Agent 2 — Code Generator + Executor (Worker)

Role: Receives the context package. Generates a transformation function. Executes it in isolation. Returns the result.

Has access to:

- Context package from Agent 1 only
- A Docker sandbox with a restricted Python environment (no network, no file system mount, whitelisted libraries only: `re`, `pandas`, `datetime`, standard builtins)

Has NO access to:

- File system
- Internet
- Raw data beyond the sample rows in the context package

Does three things in sequence:

Step 1 — Generate Produces a self-contained Python function based on the context package:

```python
def transform_price_usd(value):
    """Convert price strings to float USD. Returns None on failure."""
    if value is None or str(value).strip() == "":
        return None
    import re
    cleaned = re.sub(r'[^\d.]', '', str(value))
    try:
        return round(float(cleaned), 2)
    except ValueError:
        return None
```

Step 2 — Validate in sandbox Executes the function against the sample values from the context package inside the Docker container. Checks:

- Does it run without error?
- Does output type match `target_type`?
- Does it handle nulls correctly?
- Does it produce sensible output on the sample values?

If validation fails → Agent 2 self-corrects (re-generates, max 2 retries) before returning to Agent 1.

Step 3 — Return Returns to Agent 1:

```json
{
  "function_name": "transform_price_usd",
  "function_code": "def transform_price_usd(value)...",
  "validation_results": {
    "passed": true,
    "sample_outputs": {"$1.99": 1.99, "2.50 USD": 2.5, "null": null},
    "retries": 0
  },
  "suggested_registry_key": "string_to_float_currency"
}
```

The Function Registry (Persistent Memory)

Generated functions are not discarded. Agent 1 saves them to a function registry — a structured store of reusable transformation functions.

Registry structure:

```
function_registry/
├── registry.json              ← index of all saved functions
└── functions/
    ├── string_to_float_currency.py
    ├── extract_allergens_keyword.py
    ├── normalize_unit_ml_to_oz.py
    └── ...
```

registry.json entry:

```
{
  "key": "string_to_float_currency",
  "function_name": "transform_price_usd",
  "file": "functions/string_to_float_currency.py",
  "created_for_domain": "pricing",
  "source_type": "string",
  "target_type": "float",
  "tags": ["currency", "price", "string_to_float"],
  "used_count": 3,
  "last_used": "2026-04-08",
  "validation_passed": true
}
```

On every new pipeline run, Agent 1 checks the registry first. A function tagged `["currency", "price"]` gets reused for any future pricing domain — zero Agent 2 call, zero cost.

Over time the registry becomes a **domain-specific transformation library** that the platform built itself.

Full Workflow

```
USER CONNECTS NEW DATA SOURCE
           │
           ▼
    ┌──────────────────────────────────────────┐
    │           AGENT 1 (Orchestrator)         │
```

```
     │                                          │
     │  1. Read incoming schema + sample rows   │
     │  2. Read output schema                   │
     │  3. Read user requirements               │
     │  4. Diff → identify transformation gaps  │
     │  5. Check function registry per gap      │
     └──────────────┬───────────────────────────┘
                    │
                    │
          ┌─────────┴──────────┐
          │                    │
    [Gap covered        [Gap NOT covered
     by registry]        by registry]
          │                    │
          ▼                    ▼
    Reuse saved       Build context package
    function          Send to Agent 2
    (free, instant)          │
                             ▼
               ┌──────────────────────────┐
               │   AGENT 2 (Worker)       │
               │   [Docker Sandbox]       │
               │                          │
               │  1. Generate function    │
               │  2. Execute on samples   │
               │  3. Validate output      │
               │  4. Self-correct if      │
               │     validation fails     │
               │     (max 2 retries)      │
               └──────────────┬───────────┘
                              │
                              ▼
               ┌──────────────────────────┐
               │   AGENT 1 (receives)     │
               │                          │
               │  1. Accept function      │
               │  2. Save to registry     │
               │  3. Wire into pipeline   │
               │  4. Run on full dataset  │
               └──────────────┬───────────┘
                              │
                              ▼
               ┌──────────────────────────┐
               │   HITL REVIEW            │
               │                          │
               │  Show user:              │
               │  - What gap was found    │
```

```
              │  - Function generated  │
              │  - Validation results  │
              │  - Sample I/O preview  │
              │  [Approve / Reject]    │
              └────────────────────────┘
                           │
                           │
                           ▼
              Pipeline executes with
              new function in place
```

Trigger Conditions

The agent workflow fires when Agent 1 detects any of:

Trigger	Example
Type mismatch between source and output schema	Source: `price = "$1.99"` / Output: `price_usd: float`
New column required that source doesn't have	Output schema requires `allergens`, source has only `ingredients`
Column exists but format differs from output schema	Source: `"500ML"` / Output: `quantity_ml: int`
Domain switch with no registry coverage	First pricing source — no currency functions exist yet

---

Isolation Model (Security)

Agent 2 runs entirely inside a Docker container with:

- No network access (`--network none`)
- No file system mount (context passed as in-memory JSON only)
- Whitelisted Python libraries only (`re` , `pandas` , `datetime` , `math` , `json`)
- CPU + memory limits (prevent runaway loops)
- Execution timeout (5 seconds per validation run)

The generated function code is inspected by Agent 1 before registration:

- Static analysis: reject if any banned patterns present (`import os` , `subprocess` , `exec` , `eval` , `open(`)
- If inspection fails → Agent 1 requests regeneration with explicit constraint added to context

---

What Makes This "Truly Dynamic"

Static Pipeline	This Pipeline
New data source → developer writes new transform function	New data source → agents generate the function
Functions hardcoded for known domains	Functions generated on demand for any domain
Reuse requires developer knowledge of codebase	Reuse is automatic via registry lookup
Pipeline breaks silently on schema drift	Agent 1 detects drift and patches it
Cost: developer time per new domain	Cost: one Agent 2 call per novel transformation

---

sample execution

POC4: Dynamic ETL Pipeline - Complete Summary
Purpose
Demonstrates a schema-driven ETL pipeline powered by two AI agents that:

1. Establishes a unified schema from the first data source (USDA)
2. Detects schema gaps when processing a second source (FDA)
3. Automatically generates transformation functions via LLM (DeepSeek)
4. Produces unified output matching the established schema

Architecture

```
┌─────────────────────────────────────────────────────────┐
│ demo.py │
│ (Entry Point) │
└─────────────────────────────────────────────────────────┘
│
```

```
▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ agent1.py │
│ Agent1Orchestrator (Orchestrator) │
│ • analyze_source() - Detect schema from DataFrame │
│ • diff_schemas() - Find gaps between source & unified │
│ • check_registry() - Look for saved transforms │
│ • generate_for_gap() - Call Agent 2 for new transforms │
│ • derive_unified_schema() - Create unified schema from output │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ agent2.py │
│ Agent 2 (Worker - LLM Code Generator) │
│ • generate_transform() - Prompt DeepSeek for Python code │
│ • _validate_in_subprocess() - Sandbox execution with 5s limit │
│ • _is_code_safe() - Block dangerous patterns │
│ • _suggest_tags() - Auto-tag for registry lookup │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ registry.py │
│ FunctionRegistry │
│ • save() - Save generated .py files │
│ • load_function() - Load via importlib.util │
│ • lookup() - Find transforms by type + tags │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
│
▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│ pipeline_runner.py │
```

```
│ run_pipeline() │
│ 1. Apply LLM-generated transforms │
│ 2. Run pre-built blocks: strip_whitespace → lowercase → DQ │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

File Structure

```
POC4/
├── .env # DEEPSEEK_API_KEY
├── requirements.txt # pandas, openai, python-dotenv
└── pipeline/
├── __init__.py
├── demo.py # 230 lines - Entry point, orchestrates both runs
├── agent1.py # 244 lines - Orchestrator logic
├── agent2.py # 245 lines - LLM code generator
├── registry.py # 81 lines - Function storage/retrieval
├── pipeline_runner.py # 55 lines - Block execution engine
├── blocks/
│ ├── __init__.py
│ ├── strip_whitespace.py # Trim whitespace from strings
│ ├── lowercase_fields.py # Normalize to lowercase
│ ├── dq_score.py # Calculate DQ% based on required fields
│ └── drop_nulls_required.py # Remove rows with nulls in required cols
├── registry/
│ ├── registry.json # Saved function metadata
│ ├── unified_schema.json # Target schema from Run 1
│ └── functions/ # Generated .py transform files
├── output/
│ ├── usda_unified.csv # Run 1 output
│ └── fda_unified.csv # Run 2 output
└── tmp/ # Temporary files for validation
```

Execution Flow

Run 1: USDA FoodData (Establishes Schema)

1. Read CSV → detect schema (13 columns)
2. Select relevant columns: [description, brand_name, food_category, ingredients]

3. Rename to unified names: [product_name, brand_name, category, ingredients]
4. Run blocks: strip_whitespace → lowercase_fields → dq_score
5. Save unified_schema.json with 5 columns
6. Output: usda_unified.csv (175 rows, 5 columns, DQ: 98.1%)
   Run 2: FDA Recalls (Schema Gap Detection)
7. Read CSV → detect schema (18 columns)
8. Load unified_schema.json
9. Diff schemas - find 3 derivable gaps:
   - product_description → product_name (semantic match)
   - recalling_firm → brand_name (semantic match)
   - classification → category (semantic match)
10. Check registry → no existing transforms
11. Call Agent 2 (DeepSeek) for each gap
12. Validate generated code in sandbox (5s timeout)
13. Save transforms to registry/
14. Apply transforms + run blocks
15. Output: fda_unified.csv (500 rows, 5 columns, DQ: 66.7%)

Generated Transforms (from LLM)
Function Key
transform_product_name string_to_string_productname
transform_brand_name string_to_string_brandname
transform_category string_to_string_category

Key Design Decisions
Aspect Implementation
Sandbox Execution subprocess with 5s timeout (not exec/eval)
Module Loading importlib.util.spec_from_file_location() (not exec)
Security Forbidden patterns: os, sys, subprocess, open, eval, exec
Registry Lookup Type matching + tag scoring
Schema Matching Semantic column matching (e.g., recalling_firm → brand_name)
Fallback Mock transforms (.title()) if API fails
DQ Score % of required columns that are non-null

Data Sources

Source File

USDA FoodData usda_fooddata_sample.csv

FDA Recalls fda_recalls_sample.csv

Results

Metric USDA

Rows 175

Columns 5

Schema Match ✓

DQ Score 98.1%

LLM Transforms N/A

Running the Demo

```
cd POC4
pip install -r requirements.txt
python -m pipeline.demo
```

To force regeneration (clear cache):

```
rm -f pipeline/registry/functions/*.py
pipeline/registry/registry.json
pipeline/registry/unified_schema.json
python -m pipeline.demo
```